

O Cofre do Laboratório

Trilha de Atividades em GNU Octave

Estruturas condicionais e funções básicas

Disciplina: Matemática Computacional

Professor: Wosney Ramos de Souza

Curso: Engenharia de Software

Data: _____

Aluno(a): _____

Matrícula: _____

O professor que ocupava esta sala antes de nós deixou um cofre metálico no fundo do laboratório. Dentro dele está o material completo da disciplina: listas resolvidas, provas antigas e um bilhete que ninguém nunca leu.

O cofre abre com uma senha de quatro dígitos. Ele não anotou essa senha em lugar nenhum. Em vez disso, distribuiu quatro fragmentos por sistemas antigos que só funcionam em Octave, e protegeu cada fragmento com a credencial de quem tentasse recuperá-lo.

Isso significa que a sua senha não é igual à de nenhum colega. Cada um terá que abrir o próprio cofre.

1. Como funciona esta trilha

Você vai percorrer **cinco fases obrigatórias**. Cada fase recupera um pedaço da informação, e nenhuma delas faz sentido sozinha. Somente ao final os fragmentos se combinam e revelam a senha.

- Cada fase é um script próprio, salvo como `fase01.m`, `fase02.m` e assim por diante, todos na **mesma pasta**.
- Os scripts vêm com lacunas indicadas por `_____`. Seu trabalho é completá-las.
- Ao final de cada fase, anote o fragmento recuperado no quadro de registro da última página.
- Cada fase também exibe um **selo de conferência**. Compare com o quadro projetado na lousa: se o selo bater, pode seguir; se não bater, o erro está naquela fase, e não na seguinte.
- Terminou as cinco fases antes do tempo? Existe um compartimento secreto no cofre, e cinco fases opcionais que o abrem. Elas estão na Parte II.

Atenção

Antes de qualquer coisa, confirme que o Octave está apontando para a pasta onde você salvou os arquivos. Use o comando `pwd` para ver o diretório atual. Se ele não for a sua pasta, nenhum script será encontrado, e a mensagem de erro não vai dizer isso com clareza. Este é o problema número um da disciplina.

PARTE I: as cinco fases obrigatórias

2. Fase 1: Credenciamento

Nenhum dos sistemas antigos responde a um usuário desconhecido. O primeiro passo é se identificar e obter as duas chaves de acesso que abrem os terminais seguintes.

Sua missão

Substitua o valor da variável `matricula` pela sua matrícula real, sem pontos e sem traços, e execute o script. Ele calculará suas duas chaves pessoais e o seu código de investigador. Nesta fase você ainda não recupera nenhum dígito da senha. Ela apenas prepara o terreno.

Briefing técnico

O operador `mod(a, b)` devolve o **resto** da divisão de `a` por `b`. Por exemplo, `mod(17, 5)` vale 2, porque $17 = 3 \times 5 + 2$. Como o resto nunca alcança o divisor, `mod(x, 7)` sempre devolve algo entre 0 e 6. É assim que a sua matrícula, que é um número enorme, vira um parâmetro pequeno e controlado.

```

1  % =====
2  % FASE 1: Credenciamento
3  % Aluno: <seu nome>
4  % =====
5
6  clc;
7  clear;
8
9  % --- SUBSTITUA PELA SUA MATRICULA REAL (sem pontos e sem tracos) ---
10 matricula = 2023101234;
11
12 % --- Chaves de acesso derivadas da credencial ---
13 chave_a = 3 + mod(matricula, 7); % assume valores de 3 a 9
14 chave_b = 4 + mod(matricula, ____); % complete: deve resultar de 4 a 8
15 codigo = mod(matricula, 97); % identificador do investigador
16
17 % --- Relatorio de credenciamento ---
18 printf('=== TERMINAL DE ACESSO ===\n');
19 printf('Credencial informada : %d\n', matricula);
20 printf('Chave A : %d\n', chave_a);
21 printf('Chave B : %d\n', chave_b);
22 printf('Codigo investigador : %d\n', codigo);
23
24 printf('\nSELO DA FASE 1: %d\n', mod(chave_a + chave_b + codigo, 100));

```

Registre seu fragmento

Chave A: _____ Chave B: _____ Código: _____ Selo 1: _____

Você vai precisar desses três valores em todas as fases seguintes. Anote agora.

3. Fase 2: O primeiro fragmento

O primeiro terminal guarda um registro de leitura de sensor, capturado em algum momento das últimas vinte e quatro horas. O sistema antigo não informa o valor diretamente: ele classifica a leitura em uma faixa de operação, e é o número da faixa que interessa.

Sua missão

Complete a estrutura condicional que classifica a leitura e recupere o **primeiro dígito** da senha.

Briefing técnico

A estrutura condicional permite escolher entre caminhos diferentes:

```

1 if condicao_1
2     % executa se a condicao 1 for verdadeira
3 elseif condicao_2
4     % executa se a 1 for falsa e a 2 for verdadeira
5 else
6     % executa se nenhuma das anteriores for verdadeira
7 end

```

O Octave testa as condições **de cima para baixo** e para na primeira que for verdadeira. Por isso a ordem importa: se você escrever `leitura < 18` antes de `leitura < 6`, a segunda condição nunca será alcançada. Os operadores de comparação disponíveis são `<`, `>`, `<=`, `>=`, `==` e `~=`.

```

1 % =====
2 % FASE 2: Recuperacao do primeiro fragmento
3 % =====
4
5 clc;
6 clear;
7
8 matricula = 2023101234; % use a MESMA matricula da fase 1
9
10 % --- Leitura recuperada do terminal ---
11 leitura = mod(matricula, 24); % resultado entre 0 e 23
12 printf('Leitura recuperada: %d\n', leitura);
13
14 % --- Classificacao por faixa de operacao ---
15 % Faixa madrugada (0 a 5) -> nivel 3
16 % Faixa manha (6 a 11) -> nivel 5
17 % Faixa tarde (12 a 17) -> nivel 7
18 % Faixa noite (18 a 23) -> nivel 9
19
20 if leitura < 6
21     nivel = 3;
22 elseif leitura < 12
23     nivel = ____;
24 elseif leitura < ____
25     nivel = 7;
26 else
27     nivel = ____;
28 end
29
30 digito1 = nivel;
31
32 printf('Nivel identificado: %d\n', nivel);
33 printf('\n>>> FRAGMENTO 1 DE 4 RECUPERADO: %d\n', digito1);
34 printf('SELO DA FASE 2: %d\n', mod(leitura * 10 + digito1, 100));

```

Atenção

Repare que o bloco termina com `end`, e não com `endif`. As duas formas funcionam no Octave,

mas escolha uma e mantenha o padrão em todo o arquivo. Note também que a atribuição usa um sinal de igual, enquanto a comparação usa dois. Trocar um pelo outro dentro de um `if` é o erro mais comum desta fase.

Registre seu fragmento

Dígito 1: _____

Selo 2: _____

4. Fase 3: O registro corrompido

O segundo terminal está parcialmente danificado. Em vez de uma leitura, ele devolve três, capturadas por sensores diferentes. Cada uma precisa passar pela mesma classificação da fase anterior, e só a soma dos três níveis libera o próximo fragmento.

Sua missão

Classifique as três leituras e recupere o **segundo dígito**. Use o mesmo bloco condicional da fase 2, copiado e adaptado para cada leitura.

Ao terminar, responda por escrito à pergunta que está no final da fase.

```

1  % =====
2  % FASE 3: Tres leituras, tres classificacoes
3  % =====
4
5  clc;
6  clear;
7
8  matricula = 2023101234;
9  chave_b   = 4 + mod(matricula, 5); % recuperada da fase 1
10
11 % --- Tres leituras dos sensores danificados ---
12 leitura_a = mod(matricula, 11);
13 leitura_b = mod(matricula, 13);
14 leitura_c = mod(matricula, 17);
15
16 printf('Leituras: %d, %d, %d\n', leitura_a, leitura_b, leitura_c);
17
18 % --- Classificacao da leitura A ---
19 if leitura_a < 6
20     nivel_a = 3;
21 elseif leitura_a < 12
22     nivel_a = 5;
23 elseif leitura_a < 18
24     nivel_a = 7;
25 else
26     nivel_a = 9;
27 end
28
29 % --- Classificacao da leitura B ---
30 % COMPLETE: repita a estrutura acima, trocando leitura_a por leitura_b
31 % e nivel_a por nivel_b
32
33
34
35
36 % --- Classificacao da leitura C ---
37 % COMPLETE: repita novamente, agora para leitura_c e nivel_c
38

```

```

39
40
41
42 % --- Consolidacao ---
43 soma = nivel_a + nivel_b + nivel_c;
44 digito2 = mod(soma + chave_b, 10);
45
46 printf('Níveis: %d, %d, %d\n', nivel_a, nivel_b, nivel_c);
47 printf('Soma dos níveis: %d\n', soma);
48 printf('\n>>> FRAGMENTO 2 DE 4 RECUPERADO: %d\n', digito2);
49 printf('SELO DA FASE 3: %d\n', mod(soma * 3 + digito2, 100));

```

Briefing técnico

Pare e responda antes de seguir. Conte quantas linhas do seu script fazem exatamente a mesma coisa, mudando apenas o nome da variável. Se amanhã o critério da faixa da tarde mudar de 17 para 16, quantos lugares diferentes você teria que editar? E se fossem trinta sensores em vez de três?

Escreva sua resposta aqui:

Registre seu fragmento

Soma dos níveis: _____

Dígito 2: _____

Selo 3: _____

5. Fase 4: O bilhete do professor

Preso à lateral do terceiro terminal existe um bilhete manuscrito:

“Se você chegou até aqui copiando o mesmo bloco três vezes, parabéns pela paciência, mas eu não fiz assim. Eu ensinei o computador a classificar uma leitura, uma única vez, e depois pedi que ele fizesse isso quantas vezes eu quisesse. Guarde a receita em um arquivo com o nome dela. O terminal seguinte só aceita respostas vindas de uma receita.”

Sua missão

Você vai criar a sua primeira função. Ela recebe uma leitura e devolve o nível correspondente, exatamente como o bloco que você repetiu na fase anterior.

São dois arquivos: a função em `classifica_leitura.m` e o script em `fase04.m`, salvos na mesma pasta.

Briefing técnico

Uma função tem esta forma:

```

1 function saida = nome_da_funcao(entrada)
2     saida = alguma_coisa_calculada_com_a_entrada;
3 end

```

Três regras que resolvem quase todos os problemas desta fase. Primeiro, o arquivo precisa ter exatamente o mesmo nome da função, com extensão `.m`. Segundo, a variável de saída declarada no cabeçalho precisa receber um valor em algum caminho do corpo da função, ou o Octave reclama. Terceiro, as variáveis criadas dentro da função não existem fora dela, e as de

fora não entram: a única coisa que entra é o que você passa como parâmetro.

Arquivo 1: classifica_leitura.m

```
1 % =====
2 % Funcao que classifica uma leitura em nivel de operacao
3 % Salve este arquivo com o nome exato: classifica_leitura.m
4 % =====
5
6 function nivel = classifica_leitura(leitura)
7
8     if leitura < 6
9         nivel = 3;
10    elseif leitura < 12
11        nivel = ____;
12    elseif leitura < 18
13        nivel = ____;
14    else
15        nivel = 9;
16    end
17
18 end
```

Arquivo 2: fase04.m

```
1 % =====
2 % FASE 4: A receita do professor
3 % =====
4
5 clc;
6 clear;
7
8 matricula = 2023101234;
9 chave_a = 3 + mod(matricula, 7);
10 codigo = mod(matricula, 97);
11
12 % --- Verificacao: a funcao reproduz o resultado da fase 3? ---
13 leitura_a = mod(matricula, 11);
14 leitura_b = mod(matricula, 13);
15 leitura_c = mod(matricula, 17);
16
17 soma_func = classifica_leitura(leitura_a) ...
18             + classifica_leitura(_____) ...
19             + classifica_leitura(_____);
20
21 printf('Soma obtida com a funcao : %d\n', soma_func);
22 printf('Soma anotada na fase 3 : confira no seu registro\n');
23
24 % --- Recuperacao do terceiro fragmento ---
25 leitura_codigo = mod(codigo, 24);
26 nivel_codigo = classifica_leitura(leitura_codigo);
27
28 digito3 = mod(nivel_codigo * 3 + chave_a, 10);
29
30 printf('\nLeitura do codigo : %d\n', leitura_codigo);
31 printf('Nivel do codigo : %d\n', nivel_codigo);
32 printf('\n>>> FRAGMENTO 3 DE 4 RECUPERADO: %d\n', digito3);
33 printf('SELO DA FASE 4: %d\n', mod(soma_func + nivel_codigo + digito3, 100));
```

Atenção

Se aparecer a mensagem informando que `classifica_leitura` não foi definida, verifique três coisas nesta ordem: se o arquivo foi salvo com o nome exatamente igual ao da função, se ele está na mesma pasta do script, e se o diretório atual do Octave é essa pasta. Não existe uma quarta causa.

A soma calculada pela função deve ser **idêntica** à que você anotou na fase 3. Se for diferente, sua função não reproduz o comportamento do bloco original, e o erro está nela. Reescrever um código de forma mais organizada não pode mudar o resultado: se mudou, algo quebrou.

Registre seu fragmento

Soma pela função: _____

Dígito 3: _____

Selo 4: _____

6. Fase 5: A combinação

Com três fragmentos em mãos, o painel do cofre finalmente aceita ser ligado. O quarto dígito não está escondido em terminal nenhum: ele é gerado pela combinação dos anteriores com a sua segunda chave de acesso. É o mecanismo que garante que ninguém abra o cofre com os fragmentos de outra pessoa.

Sua missão

Monte a senha completa e submeta ao painel. Use os valores que você anotou nas fases anteriores.

```

1  % =====
2  % FASE 5: Montagem da senha e abertura do cofre
3  % =====
4
5  clc;
6  clear;
7
8  matricula = 2023101234;
9  chave_b   = 4 + mod(matricula, 5);
10
11 % --- Preencha com os fragmentos que voce anotou ---
12 digito1 = ____;
13 digito2 = ____;
14 digito3 = ____;
15
16 % --- O quarto digito nasce da combinacao dos anteriores ---
17 digito4 = mod(digito1 + digito2 + digito3 + chave_b, 10);
18
19 % --- Montagem da senha de quatro digitos ---
20 senha = digito1 * 1000 + digito2 * 100 + digito3 * 10 + digito4;
21
22 printf('=== PAINEL DO COFRE ===\n');
23 printf('Fragmentos: %d %d %d %d\n', digito1, digito2, digito3, digito4);
24 printf('Senha montada: %04d\n\n', senha);
25
26 % --- Validacao final ---
27 if senha >= 1000 && senha <= 9999
28     printf('COFRE ABERTO. Senha valida: %04d\n', senha);
29 else
30     printf('SENHA RECUSADA. Revise seus fragmentos.\n');
31 end
32

```

```
33 printf('SELO FINAL: %d\n', mod(senha + matricula, 1000));
```

Briefing técnico

O operador `&&` significa “e”. A condição inteira só é verdadeira quando as duas partes forem verdadeiras ao mesmo tempo. Existe também o `||`, que significa “ou”, verdadeiro quando pelo menos uma das partes for verdadeira. E o especificador `%04d` exibe o número com quatro dígitos, completando com zeros à esquerda quando necessário.

Registre seu fragmento

Dígito 4: _____ SENHA DO COFRE: _____ Selo final: _____

PARTE II: o compartimento secreto

Cinco fases opcionais, para quem terminou a Parte I

Com o cofre aberto, você encontra o material prometido e, sob a última pasta, uma tampa de metal com um segundo painel. Este pede um único dígito. O bilhete embaixo dela diz apenas: “para quem não parou no primeiro cofre”.

Estas cinco fases são curtas e independentes entre si, exceto pela última, que reúne os resultados. Nenhuma delas bloqueia a Parte I: se você não chegar até aqui hoje, sua nota não é afetada.

7. Fase 6: O alerta duplo

O sistema aciona um alerta somente quando as **duas** chaves de acesso são maiores que 5. Complete a condição.

```
1 matricula = 2023101234;
2 chave_a = 3 + mod(matricula, 7);
3 chave_b = 4 + mod(matricula, 5);
4
5 alerta = (chave_a > 5) ____ (chave_b > 5);    % complete com && ou ||
6
7 if alerta
8     valor6 = 5;
9 else
10    valor6 = 2;
11 end
12
13 printf('Alerta: %d | Valor 6: %d\n', alerta, valor6);
```

Note que `alerta` não é um número comum: ele é um valor lógico, e vale 1 para verdadeiro e 0 para falso. Verifique isso com `class(alerta)`.

8. Fase 7: Função com duas entradas

Crie o arquivo `custo_operacao.m`. A função recebe dois parâmetros e devolve o produto.

```
1 function total = custo_operacao(horas, tarifa)
2     total = ____ * ____;
3 end
```

No script da fase, calcule `valor7 = custo_operacao(chave_b, chave_a)` e exiba o resultado.

9. Fase 8: Função que decide

Crie o arquivo `dentro_faixa.m`. Ela recebe um valor e devolve 1 se ele estiver entre 20 e 60, incluindo os extremos, e 0 caso contrário.

```
1 function ok = dentro_faixa(valor)
2     if valor >= 20 ____ valor <= 60
3         ok = 1;
4     else
5         ok = ____;
6     end
7 end
```

Calcule `valor8 = dentro_faixa(valor7)`.

10. Fase 9: Consolidação

Reúna os três resultados anteriores em um único valor.

```
1 soma_bonus = valor6 + valor7 + valor8 * 7;  
2 printf('Soma bonus: %d\n', soma_bonus);
```

11. Fase 10: O dígito secreto

```
1 digito5 = mod(soma_bonus + digito1 + digito4, 10);  
2  
3 printf('\n>>> DIGITO DO COMPARTIMENTO SECRETO: %d\n', digito5);  
4 printf('SELO BONUS: %d\n', mod(soma_bonus * 2 + digito5, 100));
```

Registre seu fragmento

Valor 6: _____ Valor 7: _____ Valor 8: _____ Soma: _____ Dígito 5: _____

Quadro de registro

Preencha à medida que avançar e entregue esta folha junto com os arquivos.

Fase	O que foi recuperado	Valor	Selo
1	Chave A / Chave B / Código		
2	Dígito 1		
3	Soma dos níveis / Dígito 2		
4	Dígito 3		
5	Dígito 4		
SENHA DO COFRE			
Dígito secreto (opcional)			

Fechamento

Responda em poucas linhas, com suas palavras.

1. Qual foi a diferença prática entre o que você fez na fase 3 e o que fez na fase 4?

2. Por que a soma calculada pela função precisava ser igual à da fase 3?

3. Cite um erro que você cometeu durante a trilha e como descobriu que era um erro.

Entrega: os arquivos .m de todas as fases e esta folha preenchida.